

Plan d'Améliorations

GMAO MVP vers Production

Objectif : Transformer le MVP en application solide, testée et déployable. **Durée totale estimée :** 25-35 jours de travail

Prérequis : MVP Jour 0-19 valide, tests manuels OK

Table des matières

1. Solidifier les Fondations (5-7 jours)
2. Fonctionnalités Métier (7-10 jours)
3. API et Ouverture (5-7 jours)
4. Interface et Expérience (5-7 jours)
5. Déploiement et Production (3-4 jours)
6. Récapitulatif global

PHASE 1 — SOLIDIFIER LES FONDATIONS

Durée estimée : 5-7 jours · Objectif : Sécuriser le code existant avant d'ajouter quoi que ce soit

Étape 1.1 — Validation côté entité (Assert)

Durée : 1 jour

Ce qui est attendu :

Les contraintes de validation sont actuellement dans les FormTypes uniquement. Si demain tu crées une API, les données arrivent sans passer par le formulaire et aucune validation ne s'applique. Il faut ajouter des contraintes directement sur les entités.

Ce que tu dois faire :

1. Ouvrir chaque entité dans `src/Entity/`
2. Ajouter l'import en haut du fichier :

```
use Symfony\Component\Validator\Constraints as Assert;
```

3. Ajouter les contraintes sur chaque propriété :

User.php :

```
#[Assert\NotBlank]
#[Assert\Email]
#[Assert\Length(max: 180)]
private ?string $email = null;

#[Assert\NotBlank]
#[Assert\Length(min: 2, max: 100)]
private ?string $nom = null;

#[Assert\NotBlank]
#[Assert\Length(min: 2, max: 100)]
private ?string $prenom = null;
```

Demande.php :

```
#[Assert\NotBlank(message: 'Le titre est obligatoire.')]
#[Assert\Length(max: 255)]
private ?string $titre = null;

#[Assert\NotBlank(message: 'La description est obligatoire.')]
private ?string $description = null;

#[Assert\NotNull]
private ?Priorite $priorite = null;
```

Site.php :

```
#[Assert\NotBlank]
#[Assert\Length(max: 255)]
private ?string $nom = null;

#[Assert\Length(max: 10)]
private ?string $codePostal = null;
```

- **Appliquer la même logique pour :** Batiment, Equipement, CategorieEquipement, Intervention, Photo, Organisation

4. Pour chaque entité : vérifier que les champs NOT NULL en base ont bien #[Assert\NotBlank] ou #[Assert\NotNull]

Comment vérifier que c'est OK :

```
# Vérifier que le container compile toujours
php bin/console lint:container

# Tester via le code : dans un controller temporaire
$demande = new Demande(); // Vide
$errors = $validator->validate($demande);
dump($errors); // Doit lister les violations
```

✓ **Critère de validation : Aucun formulaire ne peut être soumis avec des champs obligatoires vides. Les messages d'erreur sont en français.**

Étape 1.2 — Tests automatisés unitaires

Durée : 2-3 jours

Ce qui est attendu :

Chaque service métier doit avoir des tests qui vérifient son comportement. Si tu casses quelque chose plus tard, les tests te préviennent immédiatement.

Ce que tu dois faire :

1. Installer PHPUnit :

```
composer require --dev phpunit/phpunit symfony/test-pack
```

2. Créer la structure de tests :

```
tests/
  Unit/
    Service/
      NumberingServiceTest.php
      InterventionServiceTest.php
      FileUploadServiceTest.php
    Enum/
      PrioriteTest.php
      StatutDemandeTest.php
  Functional/
    Controller/
      SecurityControllerTest.php
      DemandeControllerTest.php
      InterventionControllerTest.php
```

3. Test NumberingService :

```
class NumberingServiceTest extends TestCase
{
    // Test 1 : Le premier numéro de l'année est DEM-2026-0001
    public function testPremierNumero(): void

    // Test 2 : Le deuxième numéro incrémente DEM-2026-0002
    public function testIncrementNumero(): void

    // Test 3 : Le préfixe INT- fonctionne aussi
    public function testPrefixeIntervention(): void
}
```

4. Test InterventionService :

```
class InterventionServiceTest extends TestCase
{
    // Test 1 : Démarrer une intervention PLANIFIE → EN_COURS
    public function testDemarrerOk(): void

    // Test 2 : Démarrer une intervention déjà EN_COURS → exception
    public function testDemarrerDejaEnCours(): void

    // Test 3 : Terminer sans compte rendu → exception
    public function testTerminerSansCompteRendu(): void

    // Test 4 : Terminer avec CR → TERMINEE + dureeMinutes calculée
    public function testTerminerOk(): void

    // Test 5 : Terminer la dernière intervention → demande CLOTURE
    public function testCascadeClotureDemande(): void
}
```

5. Test Enum :

```
class PrioriteTest extends TestCase
{
    public function testTousCasesOntUnLabel(): void
    {
        foreach (Priorite::cases() as $case) {
            $this->assertNotEmpty($case->label());
        }
    }
}
```

6. Tests fonctionnels (SecurityControllerTest) :

```
class SecurityControllerTest extends WebTestCase
{
    // Test 1 : /demande sans login → redirect /login
    public function testAccesNonConnecte(): void

    // Test 2 : Login avec mauvais MDP → erreur
    public function testLoginMauvaisMdp(): void

    // Test 3 : Login OK → redirect /
    public function testLoginOk(): void
}
```

Comment vérifier que c'est OK :

```
# Lancer tous les tests
php bin/phpunit

# Lancer un test spécifique
php bin/phpunit tests/Unit/Service/InterventionServiceTest.php

# Résultat attendu : tous les tests en vert
# OK (15 tests, 30 assertions)
```

✓ **Critère de validation : Minimum 15 tests. 100% des tests passent en vert. Les 3 services sont couverts.**

Étape 1.3 — Build CSS avec Vite (remplacer le CDN)

Durée : 1 jour

Ce qui est attendu :

Le CDN Tailwind charge 3 Mo sur chaque page. Avec un build Vite, seul le CSS utilisé est inclus (50 Ko). C'est 60x plus léger.

Ce que tu dois faire :

1. Installer Vite + Tailwind :

```
composer require pentatrion/vite-bundle
npm install -D vite tailwindcss @tailwindcss/vite
```

2. Créer vite.config.js à la racine :

```
import { defineConfig } from 'vite';
import symfonyPlugin from 'vite-plugin-symfony';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [symfonyPlugin(), tailwindcss()],
  build: { rollupOptions: { input: { app: './assets/app.js' } } }
});
```

3. Créer assets/app.js : `import './styles/app.css';`

4. Créer assets/styles/app.css : `@import "tailwindcss";`

5. Dans base.html.twig, remplacer le CDN par :

```
{{ vite_entry_link_tags('app') }}
{{ vite_entry_script_tags('app') }}
```

6. Supprimer la ligne `<script src="https://cdn.tailwindcss.com">` de base.html.twig

Comment vérifier que c'est OK :

```
# Mode dev (hot reload)
npm run dev

# Build production
npm run build

# Vérifier que le dossier public/build/ contient les fichiers CSS/JS
ls public/build/
```

✓ **Critère de validation : L'app a le même rendu visuel. Le CDN Tailwind est supprimé. npm run build produit un fichier CSS < 100 Ko.**

Étape 1.4 — Sécurité : HTTPS + Headers + Rate Limiting

Durée : 1 jour

Ce qui est attendu :

Protéger l'app contre les attaques les plus courantes : brute force sur le login, injection XSS, clickjacking.

Ce que tu dois faire :

1. **Rate Limiting sur /login** : Créer config/packages/rate_limiter.yaml

```
framework:
  rate_limiter:
    login:
      policy: sliding_window
      limit: 5
      interval: '1 minute'
```

Dans SecurityController::login(), ajouter :

```
use Symfony\Component\RateLimiter\RateLimiterFactory;

public function login(RateLimiterFactory $loginLimiter, Request $request): Response
{
    $limiter = $loginLimiter->create($request->getClientIp());
    if (false === $limiter->consume(1)->isAccepted()) {
        $this->addFlash('danger', 'Trop de tentatives. Réessayez dans 1 minute.');
```

return \$this->redirectToRoute('app_login');

```
    }
    // ... reste du code
}
```

2. Headers de sécurité : Installer NelmioSecurityBundle

```
composer require nelmio/security-bundle
```

Configurer config/packages/nelmio_security.yaml :

```
nelmio_security:
    clickjacking:
        paths: { '^/.*': DENY }
    content_type:
        nosniff: true
    xss_protection:
        enabled: true
    csp:
        enabled: true
        default-src: ["'self'"]
        script-src: ["'self'", "'unsafe-inline'"]
        img-src: ["'self'", 'data:']
```

Comment vérifier que c'est OK :

```
# Test rate limiting : tenter 6 logins rapides
# Le 6ème doit afficher "Trop de tentatives"

# Vérifier les headers dans le navigateur :
# F12 > Network > cliquer sur la requête > Response Headers
# X-Content-Type-Options: nosniff
# X-Frame-Options: DENY
```

✓ **Critère de validation : 6ème tentative de login bloquée. Headers de sécurité présents dans les réponses HTTP.**

PHASE 2 — FONCTIONNALITÉS MÉTIER

Durée estimée : 7-10 jours · Objectif : Ajouter les fonctionnalités manquantes pour un produit complet

Étape 2.1 — Audit Trail (historique des actions)

Durée : 2 jours

Ce qui est attendu :

Chaque changement de statut doit être enregistré : qui a fait quoi, quand, depuis quel statut vers quel statut. Indispensable pour la traçabilité en GMAO.

Ce que tu dois faire :

1. Créer l'entité AuditLog :

```
php bin/console make:entity AuditLog
```

Champs :

- action (string 50) — "statut_change", "creation", "modification", "suppression"
- entityType (string 50) — "Demande", "Intervention"
- entityId (integer)
- oldValue (string 255, nullable) — ancien statut
- newValue (string 255, nullable) — nouveau statut
- details (text, nullable) — infos complémentaires
- user (ManyToOne vers User)
- organisation (ManyToOne vers Organisation)
- createdAt (datetime_immutable)

2. Créer le service AuditService :

```
class AuditService
{
    public function log(
        string $action,
        string $entityType,
        int $entityId,
        User $user,
        ?string $oldValue = null,
        ?string $newValue = null,
        ?string $details = null
    ): void
}
```

3. Appeler AuditService dans InterventionService :

- Après demarrerIntervention() : log "statut_change" PLANIFIE → EN_COURS
- Après terminerIntervention() : log "statut_change" EN_COURS → TERMINEE
- Après valider : log "statut_change" TERMINEE → VALIDEE

4. Appeler AuditService dans DemandeController :

- Après qualifier() : log "statut_change" A_QUALIFIER → QUALIFIE
- Après rejeter() : log "statut_change" A_QUALIFIER → REJETEE

5. Créer une page d'historique :

- Route /audit (ADMIN uniquement)
- Tableau avec : date, utilisateur, action, entité, ancien → nouveau statut
- Filtres : par entité, par utilisateur, par date

6. Migration :

```
php bin/console make:migration
php bin/console doctrine:migrations:migrate
```

Comment vérifier que c'est OK :

```
# Démarrer une intervention puis vérifier en base :
SELECT * FROM audit_log ORDER BY created_at DESC LIMIT 5;
# Doit montrer : action=statut_change, old_value=PLANIFIE, new_value=EN_COURS

# Ouvrir /audit en tant qu'admin → tableau rempli
```

✓ **Critère de validation : Chaque changement de statut génère une ligne dans audit_log. La page /audit affiche l'historique complet.**

Étape 2.2 — Notifications email

Durée : 2 jours

Ce qui est attendu :

Les utilisateurs concernés doivent être prévenus par email aux moments clés du workflow.

Ce que tu dois faire :**1. Identifier les notifications :**

Événement	Destinataire	Sujet
Intervention assignée	Technicien	"Nouvelle intervention assignée"
Intervention démarrée	Planificateur	"Intervention démarrée par [tech]"
Intervention terminée	Planificateur	"Intervention terminée, en attente de validation"
Demande clôturée	Demandeur	"Votre demande a été traitée"
Demande rejetée	Demandeur	"Votre demande a été rejetée"

2. Créer le service NotificationService :

```
class NotificationService
{
    public function __construct(
        private MailerInterface $mailer,
        private Environment $twig
    ) {}

    public function notifyInterventionAssignee(Intervention $i): void
    public function notifyInterventionDemarree(Intervention $i): void
    public function notifyInterventionTerminee(Intervention $i): void
    public function notifyDemandeCloturee(Demande $d): void
    public function notifyDemandeRejetee(Demande $d): void
}
```

3. Créer les templates email :

```
templates/email/
intervention_assignee.html.twig
intervention_demarree.html.twig
intervention_terminee.html.twig
demande_cloturee.html.twig
demande_rejetee.html.twig
```

4. Appeler NotificationService dans InterventionService et DemandeController

5. Configurer un vrai SMTP pour la production (voir DEPLOIEMENT.md)

Comment vérifier que c'est OK :

```
# En dev : Symfony intercepte les emails dans le profiler
# Ouvrir la barre de debug → icône enveloppe → vérifier le contenu

# Ou installer Mailpit (docker) :
docker run -d -p 8025:8025 -p 1025:1025 axllent/mailpit
# Configurer MAILER_DSN=smtp://localhost:1025
# Ouvrir http://localhost:8025 pour voir les emails
```

✓ **Critère de validation : Chaque action workflow envoie un email au bon destinataire. Les emails sont visibles dans le profiler Symfony.**

Étape 2.3 — Numérotation transactionnelle

Durée : 1 jour

Ce qui est attendu :

Empêcher les collisions de numéros en cas d'accès concurrent (2 personnes créent une demande en même temps).

Ce que tu dois faire :

1. Créer l'entité Counter :

```
#[ORM\Entity]
#[ORM\UniqueConstraint(fields: ['prefix', 'year'])]
class Counter
{
    #[ORM\Id] #[ORM\GeneratedValue] #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 10)]
    private string $prefix;

    #[ORM\Column]
    private int $year;

    #[ORM\Column]
    private int $lastValue = 0;
}
```

2. Modifier NumberingService avec verrou transactionnel (SELECT ... FOR UPDATE) :

```
public function generateNumero(string $prefix): string
{
    $year = (int) date('Y');
    $conn = $this->entityManager->getConnection();
    $conn->beginTransaction();
    try {
        $sql = 'SELECT last_value FROM counter WHERE prefix = ? AND year = ? FOR UPDATE';
        $result = $conn->fetchOne($sql, [$prefix, $year]);
        if ($result === false) {
            $conn->insert('counter', ['prefix'=>$prefix, 'year'=>$year, 'last_value'=>1]);
            $next = 1;
        } else {
            $next = $result + 1;
            $conn->update('counter', ['last_value'=>$next], ['prefix'=>$prefix, 'year'=>
            $year]);
        }
        $conn->commit();
    } catch (\Throwable $e) { $conn->rollBack(); throw $e; }
    return sprintf('%s-%d-%04d', $prefix, $year, $next);
}
```

3. Migration + insertion des compteurs initiaux

Comment vérifier que c'est OK :

```
# Ouvrir 2 navigateurs en parallèle
# Créer une demande dans chacun en même temps
# Vérifier que les numéros sont différents (pas de doublon)

# En base :
SELECT * FROM counter;
# Doit montrer prefix=DEM, year=2026, last_value=42
```

✓ **Critère de validation : Aucun doublon de numéro même en accès concurrent. Table counter correctement incrémentée.**

Étape 2.4 — Export PDF et Excel

Durée : 2 jours

Ce qui est attendu :

Les planificateurs veulent exporter les rapports et les listes pour les imprimer ou les envoyer par email.

Ce que tu dois faire :

1. Installer les dépendances :

```
composer require dompdf/dompdf phpoffice/phpspreadsheet
```

2. Créer ExportService :

```
class ExportService
{
    public function exportDemandesPdf(array $demandes): string
    public function exportDemandesExcel(array $demandes): string
    public function exportReportingPdf(array $kpiData): string
}
```

3. Ajouter les routes dans les controllers :

```
#[Route('/demande/export/pdf', name: 'app_demande_export_pdf')]
#[Route('/demande/export/excel', name: 'app_demande_export_excel')]
#[Route('/reporting/export/pdf', name: 'app_reporting_export_pdf')]
```

4. Créer les templates PDF (HTML simple, sans Tailwind) :

```
templates/export/
    demandes_pdf.html.twig
    reporting_pdf.html.twig
```

5. Ajouter les boutons dans les templates :

- Sur la page Demandes : boutons "Export PDF" et "Export Excel"
- Sur la page Reporting : bouton "Export PDF"

Comment vérifier que c'est OK :

```
# Cliquer sur "Export PDF" depuis la page Demandes
# Un fichier PDF se télécharge avec le tableau des demandes

# Cliquer sur "Export Excel"
# Un fichier .xlsx s'ouvre dans Excel/LibreOffice avec les bonnes colonnes
```

✓ **Critère de validation : Les PDF et Excel contiennent les bonnes données. Les filtres actifs sont pris en compte dans l'export.**

Étape 2.5 — SLA et alertes retard

Durée : 2 jours

Ce qui est attendu :

Définir un délai maximum par priorité. Si une demande dépasse ce délai, une alerte est visible sur le dashboard et un email est envoyé.

Ce que tu dois faire :

1. Définir les SLA :

Priorité	Délai max
P1 — Urgente	4 heures
P2 — Haute	24 heures
P3 — Normale	72 heures
P4 — Basse	168 heures (1 semaine)

2. Ajouter une méthode dans DemandeRepository :

```
public function findDemandesEnRetard(Organisation $org): array
```

3. Créer une commande Symfony (CRON) :

```
php bin/console make:command app:check-sla
```

La commande :

- Cherche toutes les demandes en retard
 - Envoie un email récapitulatif au planificateur
 - Exécutée toutes les heures via CRON
- 4. Ajouter un indicateur visuel sur le Dashboard :**
- Badge rouge "SLA dépassé" sur les demandes en retard
 - Compteur "Demandes en retard SLA" sur le dashboard

Comment vérifier que c'est OK :

```
# Créer une demande P1 datant de plus de 4h (modifier createdAt en base)
# Lancer la commande :
php bin/console app:check-sla
# Vérifier : email envoyé + badge visible sur le dashboard
```

✓ **Critère de validation : Les demandes en retard SLA sont identifiées. L'email contient la liste. Le dashboard les met en évidence.**

PHASE 3 — API ET OUVERTURE

Durée estimée : 5-7 jours · Objectif : Préparer l'application pour une app mobile ou des intégrations

Étape 3.1 — API REST avec API Platform

Durée : 3-4 jours

Ce qui est attendu :

Exposer les données de la GMAO via une API REST pour qu'une app mobile ou un autre système puisse lire et écrire des données.

Ce que tu dois faire :

1. Installer API Platform :

```
composer require api
```

2. Ajouter #[ApiResponse] sur les entités principales :

```
use ApiPlatform\Metadata\ApiResource;

#[ApiResponse(
    normalizationContext: ['groups' => ['demande:read']],
    denormalizationContext: ['groups' => ['demande:write']],
)]
class Demande { ... }
```

3. Configurer les groupes de sérialisation

4. Sécuriser l'API avec JWT :

```
composer require lexik/jwt-authentication-bundle
php bin/console lexik:jwt:generate-keypair
```

5. Filtrer par organisation (extension API Platform multi-tenant)

6. **Documentation auto** : API Platform génère /api/docs (Swagger)

Comment vérifier que c'est OK :

```
# Accéder à /api/docs → documentation Swagger
curl -X GET http://localhost:8000/api/demandes \
  -H "Authorization: Bearer TOKEN"
# Réponse JSON avec les demandes de l'organisation du token
```

✓ **Critère de validation** : /api/docs accessible. CRUD fonctionnel. Filtrage multi-tenant actif. JWT fonctionnel.

Étape 3.2 — Recherche avancée (fulltext MySQL)

Durée : 1-2 jours

Ce qui est attendu :

Avec des milliers de demandes, la recherche LIKE devient lente. Une recherche fulltext est nécessaire.

Ce que tu dois faire :

1. Ajouter un index FULLTEXT MySQL :

```
ALTER TABLE demande ADD FULLTEXT INDEX FT_DEMANDE_SEARCH (titre, description);
```

2. Modifier le repository :

```
public function searchFulltext(Organisation $org, string $query): array
```

✓ **Critère de validation : Recherche pertinente en moins de 200ms sur 1000+ demandes.**

PHASE 4 — INTERFACE ET EXPÉRIENCE

Durée estimée : 5-7 jours · Objectif : Améliorer l'expérience utilisateur

Étape 4.1 — Calendrier des interventions

Durée : 2-3 jours

Ce qui est attendu :

Les planificateurs veulent voir les interventions sur un calendrier mensuel/hebdomadaire.

Ce que tu dois faire :

1. Installer FullCalendar :

```
npm install @fullcalendar/core @fullcalendar/daygrid @fullcalendar/timegrid
```

2. Créer un endpoint JSON retournant les interventions
3. Créer la page calendrier avec FullCalendar
4. Ajouter le lien dans la sidebar

✓ **Critère de validation :** Calendrier affiche les interventions planifiées. Cliquer sur un événement ouvre le détail.

Étape 4.2 — QR Code sur les équipements

Durée : 1 jour

Ce qui est attendu :

Un technicien scanne le QR code collé sur un équipement et ouvre directement sa fiche pour créer une demande ou voir son historique.

Ce que tu dois faire :

1. Installer la bibliothèque :

```
composer require endroid/qrcode
```

2. Créer la route /équipement/{id}/qrcode qui génère le QR
3. Afficher et permettre l'impression sur la fiche équipement

✓ **Critère de validation :** Scanner le QR code ouvre la bonne page équipement. L'image est imprimable.

Étape 4.3 — Mode sombre

Durée : 1-2 jours

Ce que tu dois faire :

1. Utiliser les classes Tailwind dark : sur les composants
2. Bouton toggle dans le header
3. Persister le choix dans localStorage

✓ **Critère de validation : Toggle fonctionne. Couleurs cohérentes. Choix persiste après rechargement.**

PHASE 5 — DÉPLOIEMENT ET PRODUCTION

Durée estimée : 3-4 jours · Objectif : Mettre en ligne l'application

Étape 5.1 — Déploiement initial

Durée : 1-2 jours

Suivre le guide docs/DEPLOIEMENT.md pour déployer sur Railway.app (MVP) ou Cloudways (production).

✓ **Critère de validation : Application accessible en ligne. HTTPS actif. Login et upload fonctionnels.**

Étape 5.2 — Monitoring et logs

Durée : 1 jour

Ce que tu dois faire :

1. Configurer Monolog pour la production :

```
monolog:
  handlers:
    main:
      type: rotating_file
      path: '%kernel.logs_dir%/%kernel.environment%.log'
      level: warning
      max_files: 14
```

2. Sentry (gratuit pour petits projets) :

```
composer require sentry/sentry-symfony
```

✓ **Critère de validation : Erreurs visibles dans Sentry. Logs archivés sur 14 jours.**

Étape 5.3 — Backup automatique

Durée : 0.5 jour

Ce que tu dois faire :

1. Script de backup MySQL (cron quotidien à 2h du matin)
2. Conservation des 30 derniers backups

✓ **Critère de validation : Backup créé chaque nuit. 30 derniers conservés.**

Récapitulatif global

Phase	Durée	Étapes clés
Phase 1 — Fondations	5-7 jours	Assert, Tests, Vite, Sécurité
Phase 2 — Métier	7-10 jours	Audit trail, Notifications, Numérotation, Export, SLA
Phase 3 — API	5-7 jours	API REST, JWT, Recherche avancée
Phase 4 — Interface	5-7 jours	Calendrier, QR Code, Dark mode
Phase 5 — Production	3-4 jours	Deploy, Monitoring, Backup
TOTAL	25-35 jours	

Ordre recommandé

Semaine 1-2 : Phase 1 (fondations) – Assert, Tests, Vite, Sécurité
Semaine 3-4 : Phase 2 (métier) – Audit, Notifs, Export, SLA
Semaine 5 : Phase 5 (déployer en ligne) – v1 en production !
Semaine 6-7 : Phase 3 (API) – API REST + JWT
Semaine 8 : Phase 4 (interface) – Calendrier, QR, Dark mode

Conseil : Déployer en semaine 5 (avant Phase 3 et 4) permet d'avoir une version en ligne rapidement et de recueillir des retours utilisateurs pendant que tu continues à développer.